LRN Report Snapshots – Parquet Storage Implementation

Oct 20, 2026 · @jameel

Purpose and scope

Each week's standard report output is saved as Parquet files in Azure Blob, so the lab databases keep only the current week while users can still open the last 5 weeks (and 5 month-ends) of every report.

- **Problem:** the weekly Claim Level, LIS and Line Level files are cumulative. Keeping 5 weeks of raw rows in SQL Server stores roughly 5 copies of the same claims.
- **Decision:** snapshot the *report result sets*, not the raw rows, as Parquet files. A small index in LRNMaster records which snapshots exist.
- **In scope:** the Parquet file format, how files are written by `LRN.MasterFileProcessorWorker`, how they are read and shown in `LabMetricsDashboard`, and how to inspect them as a developer.
- **Out of scope:** changes to the weekly ETL itself, and the raw-table archive (optional, same mechanism).

Requirement	How Parquet meets it
Keep lab DBs small	Snapshots live in Blob, not SQL
Exact money and dates	Typed columns: decimal(p,s), date, int, string
Fast large grids	DuckDB filters, sorts and pages on the file
Reuse existing screens	DuckDB returns a `DbDataReader`, same as `SqlDataReader`
Low storage cost	Columnar encoding + Snappy compression

How Parquet stores data

Parquet is a binary, column-oriented file: values of one column are stored together, encoded, then compressed. It cannot be opened in Notepad; it is read with a Parquet library or DuckDB.

Row-wise vs column-wise

Example Denial report result:

PayerName	DeniedAmount	ClaimCount
Aetna	1200.50	4
Cigna	800.00	2
Aetna	300.25	1

JSON/CSV write it row by row. Parquet writes it column by column:

```
PayerName    -> Aetna, Cigna, Aetna
DeniedAmount -> 1200.50, 800.00, 300.25
ClaimCount   -> 4, 2, 1
```

Inside the file

1. **Row group** — a horizontal slice of rows (we use 100,000 rows per group).
2. **Column chunk** — one column's values inside a row group.
3. **Page** — the unit a column chunk is split into for encoding and compression.
4. **Footer** — the schema (column names, types, nullability), row counts, and min/max statistics per column per row group.

Encoding and compression

Technique	What it does	Effect on report data
Dictionary encoding	Stores each distinct value once; rows keep a small index (Aetna=0, Cigna=1)	Payer, clinic, CPT and status columns shrink sharply

Technique	What it does	Effect on report data
Run-length encoding (RLE)	Stores repeats as value × count	Sorted or repeated columns shrink further
Snappy compression	Fast general compression on each page	**Default** — fastest writes and reads
Zstd compression	Stronger compression, more CPU	Use when storage size matters more than write speed

Why reads are fast

- A query reads only the columns it selects.
- Footer min/max statistics let DuckDB skip row groups that cannot match a filter.
- Types are stored in the file, so no parsing of strings into numbers or dates.

Storage design

One Parquet file is written per lab, per weekly run, per report, per result set, at a deterministic Blob path so a re-run overwrites the same file.

Blob layout

- Container: `report-snapshots` (private, no public access).
- Path: `{labCode}/{yyyy}/{weekFolder}/{runId}/{reportCode}/{resultSetName}.v{schemaVersion}.parquet`
- Example: `NWL/2026/Week_41/RUN_20261012/DENIAL/Summary.v1.parquet`
- Content type: `application/vnd.apache.parquet`; blob metadata `sha256` = file hash.

Snapshot grain

The file holds the report at the grain the UI filters and pages on (e.g. per payer, clinic, month), not a result pre-filtered by one user's selection. User filters are applied later by DuckDB.

SQL Server to Parquet type mapping

SQL Server type	.NET type read	Parquet column	Note
int, smallint, tinyint, bigint	int / short / byte / long	INT32 / INT64, nullable	Value types always written nullable
decimal(p,s), numeric, money	decimal	DECIMAL(p,s)	Precision/scale taken from the reader schema; money → (19,4)
float, real	double / float	DOUBLE / FLOAT	
bit	bool	BOOLEAN	
date, datetime, datetime2, smalldatetime	DateTime	TIMESTAMP	
datetimeoffset	DateTimeOffset	TIMESTAMP (UTC)	Converted to UTC DateTime
varchar, nvarchar, char, nchar	string	BYTE_ARRAY (UTF8)	Dictionary-encoded automatically
uniqueidentifier	Guid	BYTE_ARRAY (UTF8)	Written as string
varbinary	byte[]	BYTE_ARRAY	Avoid in report outputs
sql_variant	object	BYTE_ARRAY (UTF8)	Written as string

Unnamed SP columns become `Col{n}`; duplicate names get a `_1`, `_2` suffix.

Settings

Setting	Value	Reason
Compression	Snappy	Fast; switch to Zstd per report if size matters

Setting	Value	Reason
Row group size	100,000 rows	Good balance of memory use and skip efficiency
Files per result set	1	Report outputs are small; raw archives may split by row group only
Schema version	In file name and index table	Old snapshots stay readable after report changes

Writing Parquet in the worker

After a lab's weekly run is marked Success, the worker runs each report's SP, streams the rows into a local Parquet file, validates it, uploads it to Blob and records it in LRNMaster.

Packages

```
dotnet add package Parquet.Net
dotnet add package DuckDB.NET.Data.Full
dotnet add package Azure.Storage.Blobs
dotnet add package Azure.Identity
dotnet add package Microsoft.Data.SqlClient
```

Put the writer, reader and blob store in a shared class library `LRN.Snapshots`, referenced by both `LRN.ReportEngine` and `LabMetricsDashboard`.

Write steps per report

1. Execute the report SP against the lab DB with `CommandBehavior.SequentialAccess`.
2. For each result set, build the Parquet schema from `reader.GetColumnSchema()`.
3. Buffer 100,000 rows per column into typed arrays, then write one row group.
4. Close the file (writes the footer), then re-count rows with DuckDB.
5. Hash (SHA-256) and upload to the deterministic Blob path, overwriting.
6. Insert a `ReportSnapshotFile` row: path, row count, size, hash, schema JSON, schema version.

Writer: SqlDataReader to Parquet

```
using System.Data.Common;
using System.Text.Json;
using Parquet;
using Parquet.Data;
using Parquet.Schema;

public sealed record ParquetWriteResult(long RowCount, string SchemaJson);

public static class ParquetSnapshotWriter
{
    public static async Task<ParquetWriteResult> WriteAsync(
        DbDataReader reader, string filePath, int rowGroupSize,
CompressionMethod compression, CancellationToken ct)
    {
        var columns = reader.GetColumnSchema();
        var fields = new List<DataField>();
        var elementTypes = new List<Type>();
        var converters = new List<Func<object, object>>();
        var used = new HashSet<string>(StringComparer.OrdinalIgnoreCase);

        for (int i = 0; i < columns.Count; i++)
        {
            var col = columns[i];
            var name = Unique(string.IsNullOrWhiteSpace(col.ColumnName) ?
$"Col{i}" : col.ColumnName, used);
            var (field, type, conv) = Map(name, col.DataType ??
typeof(string), col);
            fields.Add(field); elementTypes.Add(type); converters.Add(conv);
        }

        var schema = new ParquetSchema(fields.Cast<Field>().ToArray());
        long total = 0;

        await using (var fs = File.Create(filePath))
        using (var writer = await ParquetWriter.CreateAsync(schema, fs,
cancellationToken: ct))
        {
            writer.CompressionMethod = compression;          // Snappy
(default) or Zstd
            var buffers = NewBuffers(elementTypes, rowGroupSize);
            int n = 0;
```

```
            while (await reader.ReadAsync(ct))
            {
                for (int c = 0; c < fields.Count; c++)
                {
                    object v = reader.GetValue(c);
                    buffers[c].SetValue(v is DBNull ? null : converters[c]
(v), n);
                }
                if (++n == rowGroupSize)
                {
                    await WriteGroupAsync(writer, fields, buffers, n, ct);
                    total += n; n = 0;
                    buffers = NewBuffers(elementTypes, rowGroupSize);
                }
            }

            if (n > 0 || total == 0)                           // empty report
still gets a valid file
            {
                await WriteGroupAsync(writer, fields, buffers, n, ct);
                total += n;
            }
        }

        var schemaJson = JsonSerializer.Serialize(fields.Select((f, i) =>
            new { name = f.Name, type = elementTypes[i].Name, sqlType =
columns[i].DataTypeName }));
        return new ParquetWriteResult(total, schemaJson);
    }

    private static Array[] NewBuffers(List<Type> types, int size) =>
        types.Select(t => Array.CreateInstance(t, size)).ToArray();

    private static async Task WriteGroupAsync(ParquetWriter writer,
List<DataField> fields, Array[] buffers, int count, CancellationToken ct)
    {
        using ParquetRowGroupWriter group = writer.CreateRowGroup();
        for (int c = 0; c < fields.Count; c++)
        {
            Array data = buffers[c];
            if (data.Length != count)
            {
```

```
                var trimmed =
Array.CreateInstance(data.GetType().GetElementType()!, count);
                Array.Copy(data, trimmed, count);
                data = trimmed;
            }
            await group.WriteColumnAsync(new DataColumn(fields[c], data),
ct);
        }
    }

    private static (DataField, Type, Func<object, object>) Map(string name,
Type src, DbColumn col)
    {
        static object Same(object v) => v;

        if (src == typeof(Guid))           return (new DataField<string>
(name), typeof(string), v => v.ToString()!);
        if (src == typeof(char))           return (new DataField<string>
(name), typeof(string), v => v.ToString()!);
        if (src == typeof(object))         return (new DataField<string>
(name), typeof(string), v => Convert.ToString(v)!);
        if (src == typeof(DateTimeOffset)) return (new DataField<DateTime?>
(name), typeof(DateTime?), v => ((DateTimeOffset)v).UtcDateTime);

        if (src == typeof(decimal))
        {
            int p = col.NumericPrecision is > 0 and <= 38 ?
col.NumericPrecision.Value : 38;
            int s = col.NumericScale is >= 0 && col.NumericScale <= p ?
col.NumericScale.Value : 4; // money reports scale 255
            return (new DecimalDataField(name, p, s, isNullable: true),
typeof(decimal?), Same);
        }

        if (src == typeof(string) || src == typeof(byte[]))
            return (new DataField(name, src), src, Same);

        var nullable = typeof(Nullable<>).MakeGenericType(src);   // int ->
int?, DateTime -> DateTime?
        return (new DataField(name, nullable), nullable, Same);
    }

    private static string Unique(string name, HashSet<string> used)
```

```
    {
        var candidate = name;
        for (int i = 1; !used.Add(candidate); i++) candidate = $"{name}_{i}";
        return candidate;
    }
}
```

Validate, upload and register

```
// 1. Validate: row count in the file must equal rows written
await using (var duck = new DuckDBConnection("DataSource=:memory:"))
{
    await duck.OpenAsync(ct);
    await using var cmd = duck.CreateCommand();
    cmd.CommandText = $"SELECT COUNT(*) FROM
read_parquet('{localPath.Replace("'", "''")}')";
    long actual = Convert.ToInt64(await cmd.ExecuteScalarAsync(ct));
    if (actual != result.RowCount)
        throw new InvalidOperationException($"Row count mismatch
{result.RowCount} vs {actual}");
}

// 2. Hash + upload (overwrite = idempotent re-run)
string sha;
await using (var fs = File.OpenRead(localPath))
    sha = Convert.ToHexString(await SHA256.HashDataAsync(fs, ct));

var blob = container.GetBlobClient(blobPath);
await blob.UploadAsync(localPath, new BlobUploadOptions
{
    HttpHeaders = new BlobHttpHeaders { ContentType =
"application/vnd.apache.parquet" },
    Metadata = new Dictionary<string, string> { ["sha256"] = sha }
}, ct);

// 3. Register in LRNMaster
await index.AddFileAsync(snapshotRunId, new SnapshotFileInfo(
    def.ReportCode, resultSetName, blobPath, result.RowCount,
    new FileInfo(localPath).Length, sha, result.SchemaJson,
def.SchemaVersion), ct);
```

Index tables (LRNMaster)

```
CREATE TABLE dbo.ReportSnapshotRun
(
    SnapshotRunId BIGINT IDENTITY(1,1) PRIMARY KEY,
    LabId         INT           NOT NULL,
    LabName       NVARCHAR(100) NOT NULL,
    RunId         NVARCHAR(64)  NOT NULL,
    WeekFolder    NVARCHAR(100) NOT NULL,
    SnapshotDate  DATE          NOT NULL,
    Status        VARCHAR(20)   NOT NULL,   -- InProgress | Success | Failed
    ErrorMessage  NVARCHAR(4000) NULL,
    CreatedOn     DATETIME2(0)  NOT NULL DEFAULT SYSUTCDATETIME(),
    CompletedOn   DATETIME2(0)  NULL,
    CONSTRAINT UQ_ReportSnapshotRun UNIQUE (LabId, RunId)
);

CREATE TABLE dbo.ReportSnapshotFile
(
    SnapshotFileId BIGINT IDENTITY(1,1) PRIMARY KEY,
    SnapshotRunId  BIGINT NOT NULL REFERENCES
dbo.ReportSnapshotRun(SnapshotRunId) ON DELETE CASCADE,
    ReportCode     VARCHAR(50)   NOT NULL,
    ResultSetName  NVARCHAR(100) NOT NULL,
    BlobPath       NVARCHAR(500) NOT NULL,
    [RowCount]     BIGINT        NOT NULL,
    SizeBytes      BIGINT        NOT NULL,
    ContentSha256  CHAR(64)      NOT NULL,
    SchemaJson     NVARCHAR(MAX) NOT NULL,
    SchemaVersion  INT           NOT NULL,
    CreatedOn      DATETIME2(0)  NOT NULL DEFAULT SYSUTCDATETIME(),
    CONSTRAINT UQ_ReportSnapshotFile UNIQUE (SnapshotRunId, ReportCode,
ResultSetName)
);
```

A snapshot failure is logged (`Status = Failed` , `LRN_Error_Log`) and retried next cycle; it never fails the weekly load.

Reading Parquet

The web app downloads a snapshot file once to a local cache, then DuckDB runs SQL on it and returns a `DbDataReader` that the existing report mappers consume.

Read steps per request

1. Look up the file in `ReportSnapshotFile` by `SnapshotRunId`, `ReportCode`, `ResultSetName` and `LabId` (the lab check blocks cross-lab access).
2. Get a local copy: cache path `{cacheRoot}/{sha256}.parquet`; download from Blob only if missing.
3. Open an in-memory DuckDB connection.
4. Run `SELECT … FROM read_parquet('<path>') WHERE … ORDER BY … LIMIT … OFFSET …`.
5. Pass the reader to the report's mapper, exactly as with SQL Server.

Local file cache

Files never change after upload, so caching by hash is safe. A re-snapshot of the same run produces a new hash and a new cache entry.

```
public sealed class SnapshotFileCache(BlobContainerClient container, string
cacheRoot)
{
    private static readonly ConcurrentDictionary<string, SemaphoreSlim> Locks
= new();

    public async Task<string> GetLocalPathAsync(string blobPath, string
sha256, CancellationToken ct)
    {
        var local = Path.Combine(cacheRoot, $"{sha256}.parquet");
        if (File.Exists(local)) { File.SetLastAccessTimeUtc(local,
DateTime.UtcNow); return local; }

        var gate = Locks.GetOrAdd(local, _ => new SemaphoreSlim(1, 1));
        await gate.WaitAsync(ct);
        try
        {
            if (!File.Exists(local))
            {
                Directory.CreateDirectory(cacheRoot);
                var tmp = local + ".part";
                await container.GetBlobClient(blobPath).DownloadToAsync(tmp,
ct);
                File.Move(tmp, local, overwrite: true);
            }
```

```
            return local;
        }
        finally { gate.Release(); }
    }

    public void Purge(TimeSpan maxIdle)   // run daily from a hosted service
    {
        if (!Directory.Exists(cacheRoot)) return;
        foreach (var f in new
DirectoryInfo(cacheRoot).EnumerateFiles("*.parquet"))
            if (DateTime.UtcNow - f.LastAccessTimeUtc > maxIdle) { try {
f.Delete(); } catch { } }
    }
}
```

DuckDB query with safe filtering and paging

```
public sealed record ReportQuery(
    string ReportCode, string ResultSetName,
    IReadOnlyDictionary<string, object?> Filters,
    string? SortColumn = null, bool SortDescending = false,
    int? Page = null, int PageSize = 50);

public static class ParquetReportReader
{
    public static async Task<T> QueryAsync<T>(string parquetPath, ReportQuery
q,
        Func<DbDataReader, CancellationToken, Task<T>> map, CancellationToken
ct)
    {
        await using var con = new DuckDBConnection("DataSource=:memory:");
        await con.OpenAsync(ct);

        var columns = await ColumnsAsync(con, parquetPath, ct);   //
whitelist from the file schema
        await using var cmd = con.CreateCommand();

        var where = new List<string>();
        foreach (var (col, val) in q.Filters)
        {
            if (!columns.Contains(col)) continue;                  // unknown
column ignored
```

```
            if (val is null) { where.Add($"{Quote(col)} IS NULL"); continue;
}
            where.Add($"{Quote(col)} = ?");
            cmd.Parameters.Add(new DuckDBParameter(val));          // values
always parameterised
        }

        var sql = $"SELECT * FROM read_parquet('{Esc(parquetPath)}')";
        if (where.Count > 0) sql += " WHERE " + string.Join(" AND ", where);
        if (q.SortColumn is not null && columns.Contains(q.SortColumn))
            sql += $" ORDER BY {Quote(q.SortColumn)} {(q.SortDescending ?
"DESC" : "ASC")}";
        if (q.Page is int p)
            sql += $" LIMIT {q.PageSize} OFFSET {(Math.Max(p, 1) - 1) *
q.PageSize}";

        cmd.CommandText = sql;
        await using var reader = await cmd.ExecuteReaderAsync(ct);
        return await map(reader, ct);
    }

    private static async Task<HashSet<string>> ColumnsAsync(DuckDBConnection
con, string path, CancellationToken ct)
    {
        var set = new HashSet<string>(StringComparer.OrdinalIgnoreCase);
        await using var cmd = con.CreateCommand();
        cmd.CommandText = $"DESCRIBE SELECT * FROM
read_parquet('{Esc(path)}')";
        await using var r = await cmd.ExecuteReaderAsync(ct);
        while (await r.ReadAsync(ct)) set.Add(r.GetString(0));
        return set;
    }

    private static string Quote(string id) => "\"" + id.Replace("\"", "\"\"")
+ "\"";
    private static string Esc(string s) => s.Replace("'", "''");
}
```

A matching `CountAsync` runs `SELECT COUNT(*)` with the same `WHERE` for pager totals.

Mapping to existing view models

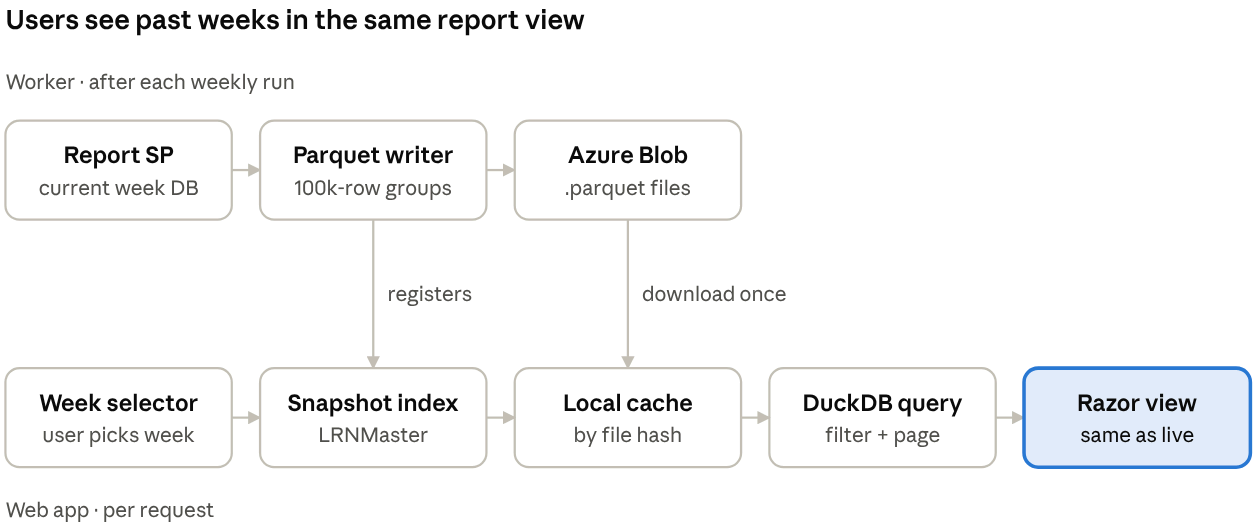
Change each mapper's parameter from `SqlDataReader` to `DbDataReader`, and read numbers with `Convert` so wider DuckDB integer types still map:

```
public static async Task<List<DenialRow>> MapAsync(DbDataReader r,
CancellationToken ct)
{
    int oPayer = r.GetOrdinal("PayerName"), oAmt =
r.GetOrdinal("DeniedAmount"), oCnt = r.GetOrdinal("ClaimCount");
    var rows = new List<DenialRow>();
    while (await r.ReadAsync(ct))
        rows.Add(new DenialRow
        {
            PayerName    = r.IsDBNull(oPayer) ? "" : r.GetString(oPayer),
            DeniedAmount = r.IsDBNull(oAmt) ? 0m :
Convert.ToDecimal(r.GetValue(oAmt)),
            ClaimCount   = r.IsDBNull(oCnt) ? 0 :
Convert.ToInt32(r.GetValue(oCnt))
        });
    return rows;
}
```

The live path and the snapshot path then share one interface, `IReportDataSource`, with a SQL Server implementation and a Parquet implementation. Filters that a live SP applies (date ranges, IN-lists) must be mirrored in the Parquet `WHERE` builder per report.

Showing in the UI

The browser never receives Parquet: the server turns the file into the same view model and Razor view the live report already uses, plus a week selector and a read-only banner.



snapshot flow · written once, read per request

The worker writes and registers each file once; every page view reads it through the cache and DuckDB into the existing view.

Controller

```
public async Task<IActionResult> DenialReport(string lab, long? snapshot, int
page = 1,
                                              string? payer = null,
CancellationToken ct = default)
{
    var labInfo = _labResolver.Resolve(lab);
    IReportDataSource source = snapshot is null
        ? _live.Create(labInfo.ConnectionString)                    // SQL
Server, today's path
        : _snapshots.Create(labInfo.LabId, snapshot.Value);         //
Parquet + DuckDB

    var filters = new Dictionary<string, object?>();
    if (payer is not null) filters["PayerName"] = payer;

    var query = new ReportQuery("DENIAL", "Summary", filters, "DeniedAmount",
true, page, 50);

    var vm = new DenialReportViewModel
    {
        Rows          = await source.ReadAsync(query, DenialMapper.MapAsync,
ct),
```

```
        TotalRows     = await source.CountAsync(query, ct),
        Page          = page,
        LabKey        = lab,
        IsSnapshot    = source.IsSnapshot,
        SnapshotLabel = source.SnapshotLabel,                          //
"Week_41 (as of 12-Oct-2026)"
        SelectedSnapshotRunId = snapshot,
        Snapshots     = await _catalog.ListAsync(labInfo.LabId, "DENIAL", ct)
    };
    return View(vm);                                                    //
same view as live
}
```

Week selector and banner (`_SnapshotSelector.cshtml`)

```
@model ISnapshotAwareViewModel
<div class="d-flex align-items-center gap-2 mb-3">
    <label for="snapshotSelect" class="form-label mb-0 fw-semibold">Data as
of</label>
    <select id="snapshotSelect" class="form-select form-select-sm w-auto"
            onchange="location.search = this.value ? '?
lab=@Model.LabKey&snapshot=' + this.value : '?lab=@Model.LabKey'">
        <option value="" selected="@(Model.SelectedSnapshotRunId is
null)">Current (live)</option>
        @foreach (var s in Model.Snapshots.DistinctBy(x => x.SnapshotRunId))
        {
            <option value="@s.SnapshotRunId"
selected="@(Model.SelectedSnapshotRunId == s.SnapshotRunId)">
                @s.WeekFolder — @s.SnapshotDate.ToString("dd MMM yyyy")
            </option>
        }
    </select>
</div>

@if (Model.IsSnapshot)
{
    <div class="alert alert-warning py-2 d-flex align-items-center gap-2"
role="status">
        <i class="bi bi-clock-history"></i>
        <span>Read-only snapshot: <strong>@Model.SnapshotLabel</strong>
</span>
```

```
        <a class="ms-auto btn btn-sm btn-outline-dark" href="?
lab=@Model.LabKey">Back to current</a>
    </div>
}
```

The dropdown is filled from `usp_ReportSnapshot_List @LabId, @ReportCode`, which returns successful snapshots newest first.

UI rules

- Hide or disable write actions (assign, status change, notes) when `IsSnapshot` is true.
- Paging and sorting go back to the server, which re-queries the Parquet file; DataTables server-side mode works unchanged.
- Excel and PDF exports use the same view model, so they export normally; add the snapshot label to the export header.
- Optionally drop the current week from the dropdown, since "Current (live)" already shows it.

Viewing files as a developer

Download the file from Blob with Azure Storage Explorer (it cannot preview Parquet), then open it with any of these tools.

Tool	How to use	Best for
DuckDB CLI	`duckdb` then `SELECT * FROM 'Summary.v1.parquet' LIMIT 20;`	Quick checks, counts, filters
VS Code	Install a Parquet viewer extension, open the file	Browsing rows while coding
Parquet Viewer (Windows app)	Open the file	Grid view without code
Python	`pandas.read_parquet("Summary.v1.parquet")`	Ad-hoc analysis
Excel / Power BI	Power Query → Get Data → Parquet	Business users checking a week

Useful DuckDB commands:

```
-- schema stored in the file
DESCRIBE SELECT * FROM 'Summary.v1.parquet';

-- row groups, compression and min/max statistics
SELECT row_group_id, path_in_schema, compression, stats_min, stats_max
FROM parquet_metadata('Summary.v1.parquet');

-- compare two weeks of the same report
SELECT a.PayerName, a.DeniedAmount AS week40, b.DeniedAmount AS week41
FROM 'Week_40/DENIAL/Summary.v1.parquet' a
JOIN 'Week_41/DENIAL/Summary.v1.parquet' b USING (PayerName);
```

Validation, retention, security and operations

Every file is checked on write, kept for 5 weekly plus 5 month-end runs per lab, stored privately because it contains PHI, and never rewritten once published.

Retention

After each successful snapshot, `usp_ReportSnapshot_GetExpired @LabId, @KeepWeekly = 5, @KeepMonthEnd = 5` returns runs outside the policy. The worker deletes their blobs first, then the `ReportSnapshotRun` row (files cascade). A run is kept if it is one of the 5 latest, or the latest run of one of the 5 latest months. Both counts come from config.

Security

Control	Setting
Access	Managed identity via `DefaultAzureCredential`; worker = Storage Blob Data Contributor, web app = Storage Blob Data Reader
Network	Private endpoint or storage firewall limited to the VM / VNet
Encryption	At rest by default; customer-managed key if compliance requires
Recovery	Blob soft delete 7–14 days

Control	Setting
Safety net	Lifecycle rule: Cool tier after 45 days, delete after 200 days (never Archive tier)
Web cache	Restricted folder ACL; purge files idle more than 7 days
Lab isolation	Every snapshot read checks `LabId` in the index query

Schema versioning

- Bump `SchemaVersion` when a report's columns change; it goes into the file name and the index row.
- Mappers tolerate missing columns (check `HasColumn`, use a default) or use a version-specific mapper.
- Never rewrite old snapshots: they show what users saw that week.

Test checklist

- [ ] Snapshot for a pilot lab creates one Success run and one file row per result set
- [ ] File row count = SP row count = DuckDB `COUNT(*)`
- [ ] Snapshot of the current week matches the live report (totals, first page, sorted page)
- [ ] Decimals keep precision and scale; dates and NULLs are correct
- [ ] Re-running the same RunId overwrites files with no duplicate index rows
- [ ] Blocking storage access fails the snapshot but not the weekly load
- [ ] After 7 weekly runs only the retained runs and their blobs remain
- [ ] Another lab's snapshot id returns 404
- [ ] Unknown filter or sort column is ignored; values are parameterised
- [ ] Second open of a snapshot is served from the cache

Rollout

1. Create the LRNMaster tables, Blob container and identity roles; build `LRN.Snapshots`.
2. Enable the writer for one pilot lab and 2–3 reports.
3. Refactor those reports' mappers to `DbDataReader` and add the selector and banner.
4. Client UAT on real past weeks, then add the remaining reports with filter parity.

5. Enable all labs; monitor file sizes, snapshot run time and cache size.

Snapshots exist only from go-live forward. To show 5 weeks on day one, reprocess the last few week folders into a scratch DB and snapshot from it, if those source files are still available.